

Filtros y Convolución

¿Qué es un filtro?

- Principal diferencia con operadores puntuales: los filtros generalmente utilizan más de un píxel de la imagen de entrada para obtener cada píxel de la imagen de salida.
- El filtrado espacial modifica una imagen reemplazando el valor de cada píxel por una función de los valores del píxel y sus vecinos.
- Si la función aplicada es lineal entonces es un filtro lineal espacial.

Ejemplo: suavizado de una imagen

- No puede obtenerse a partir de operadores puntuales
- Para determinar un píxel $I'(u, v)$ en la imagen de salida I' , utilizamos el píxel original $I(u, v) = p_0$ de la imagen de entrada I más sus vecinos $p_1, p_2, \dots, p_8$.
- Idea: reemplazar cada píxel por un promedio de píxeles vecinos

$$I'(u, v) \leftarrow \frac{p_0 + p_1 + p_2 + p_3 + p_4 + p_5 + p_6 + p_7 + p_8}{9}$$



Filtros

Matriz H del filtro

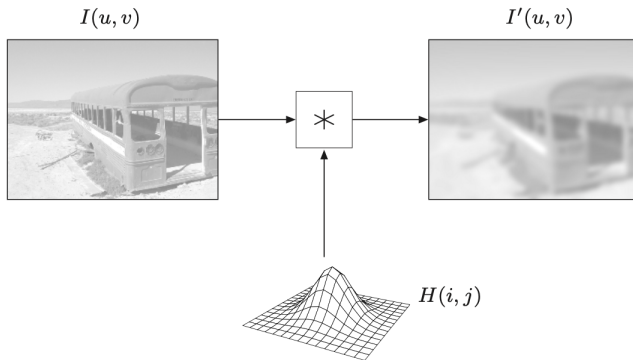
- Para cualquier filtro lineal, el tamaño y forma de la región de soporte se especifica por una matriz H del filtro.
- En nuestro ejemplo:

$$H = \frac{1}{9} \begin{pmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{pmatrix}$$

Convolución

Operador “*” de convolución: $I' = I * H$

$$I'(u, v) = \sum_{i=-\infty}^{\infty} \sum_{j=-\infty}^{\infty} I(u - i, v - j) \cdot H(i, j)$$



Convolución

Propiedades

Conmutatividad

$$H * I = I * H$$

Linealidad

$$\begin{aligned}(s \cdot I) * H &= I * (s \cdot H) = s \cdot (I * H) \\ (I_1 + I_2) * H &= (I_1 * H) + (I_2 * H) \\ (b + I) * H &\neq b + (I * H)\end{aligned}$$

Asociatividad

$$(A * B) * C = A * (B * C)$$

Filtros y funciones separables

- Una función bidimensional $G(x, y)$ se dice *separable* si puede expresarse como el producto de dos funciones unidimensionales: $G(x, y) = G_1(x) \cdot G_2(y)$.
- Una matriz de filtro H es *separable* si puede expresarse como el producto externo de dos vectores h_1 y h_2 : $H = h_1 h_2^t$

Ejemplo de nuestro filtro suavizante

$$H_{xy} = \frac{1}{9} \begin{pmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{pmatrix} = \begin{pmatrix} 1/3 \\ 1/3 \\ 1/3 \end{pmatrix} \begin{pmatrix} 1/3 & 1/3 & 1/3 \end{pmatrix} = h_y h_x$$

Propiedad: $H_{xy} = h_y h_x = h_y * h_x = h_x * h_y$

Aplicación secuencial: $I' \leftarrow (I * h_x) * h_y = I * \underbrace{(h_x * h_y^t)}_{H_{xy}}$

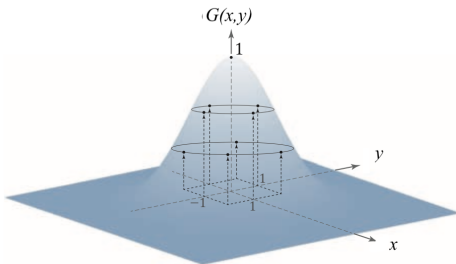
Filtro Gaussiano - *Smoothing*

Definición

- Siendo σ el ancho (desviación estándar) de la función en forma de campana y r es la distancia (radio) desde el centro:

$$G_{\sigma}(x, y) = e^{-\frac{r^2}{2\sigma^2}} = e^{-\frac{x^2+y^2}{2\sigma^2}}$$

- Matriz H^{σ} del filtro gaussiano $\rightarrow$ discretización de G .



$$\frac{1}{4.8976} \times$$

0.3679	0.6065	0.3679
0.6065	1.0000	0.6065
0.3679	0.6065	0.3679

Separabilidad del filtro gaussiano

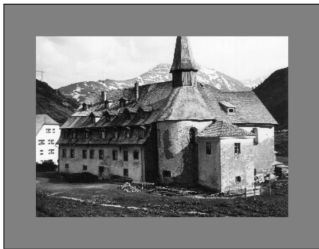
- La función que lo define es separable:

$$G_{\sigma}(x, y) = e^{-\frac{x^2+y^2}{2\sigma^2}} = e^{-\frac{x^2}{2\sigma^2}} \cdot e^{-\frac{y^2}{2\sigma^2}} = g_{\sigma}(x) \cdot g_{\sigma}(y)$$

- La convolución con un filtro gaussiano se puede implementar con dos filtros:

$$I' \leftarrow I * H^{\sigma} = I * h_x^{\sigma} * h_y^{\sigma}$$

Padding



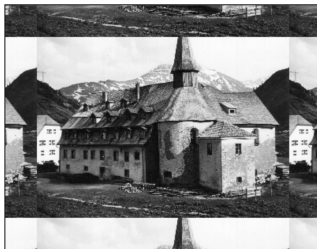
(a)



(b)



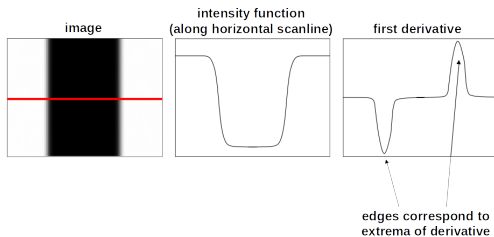
(c)



(d)

Derivadas y Bordes

- Un borde es el límite entre dos regiones con propiedades de nivel de gris relativamente distintas.
- La base de la mayoría de las técnicas de detección de bordes es el cálculo de un operador derivado local.
- La magnitud de la primera derivada calculada dentro de una vecindad alrededor del píxel de interés, se puede utilizar para detectar la presencia de un borde en una imagen.



Si consideramos una ventana de 2×2 , alrededor de cada punto (i, j) de modo que:

$$\left(\begin{array}{c|c} f(i, j) & f(i, j + 1) \\ \hline f(i + 1, j) & f(i + 1, j + 1) \end{array} \right)$$

en este caso una manera de aproximar la derivada primera es mediante:

$$\begin{aligned} \frac{\partial f}{\partial x} &= f(i + 1, j) - f(i, j) \\ \frac{\partial f}{\partial y} &= f(i, j + 1) - f(i, j) \end{aligned}$$

otra forma es haciendo diferencias cruzadas:

$$\begin{aligned} \frac{\partial f}{\partial x} &= f(i, j) - f(i + 1, j + 1) \\ \frac{\partial f}{\partial y} &= f(i, j + 1) - f(i + 1, j) \end{aligned}$$

Si en cambio son de 3×3 , alrededor del punto (i, j)

$$\left(\begin{array}{c|c|c} f(i-1, j-1) & f(i-1, j) & f(i-1, j+1) \\ \hline f(i, j-1) & f(i, j) & f(i, j+1) \\ \hline f(i+1, j-1) & f(i+1, j) & f(i+1, j+1) \end{array} \right)$$

que se puede aproximar:

$$f_x = f_{x_3} - f_{x_1} \quad f_y = f_{y_3} - f_{y_1}$$

donde:

$$f_{x_3} = f(i+1, j-1) + f(i+1, j) + f(i+1, j+1)$$

$$f_{x_1} = f(i-1, j-1) + f(i-1, j) + f(i-1, j+1)$$

$$f_{y_3} = f(i, j+1) + f(i+1, j+1) + f(i+2, j+1)$$

$$f_{y_1} = f(i, j-1) + f(i+1, j-1) + f(i+2, j-1)$$

f_{x_k} son las filas y f_{y_k} son las columnas, con $k = 1, 2, 3$.

Operadores gradientes isotrópicos

Prewitt

$$\begin{pmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{pmatrix} \begin{pmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{pmatrix}$$

Sobel

$$\begin{pmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{pmatrix}$$

Isotrópico

$$\begin{pmatrix} -1 & 0 & 1 \\ -\sqrt{2} & 0 & \sqrt{2} \\ -1 & 0 & 1 \end{pmatrix} \begin{pmatrix} -1 & -\sqrt{2} & -1 \\ 0 & 0 & 0 \\ 1 & \sqrt{2} & 1 \end{pmatrix}$$

Detector de bordes Canny

Criterios de detección de Canny

- Criterio de Detección: no perder bordes y que no haya respuestas espúreas.
- Criterio de Localización: minimizar la distancia entre la posición real y el borde detectado.
- Criterio de una sola Respuesta: minimiza múltiples respuestas múltiples correspondientes a un único borde.

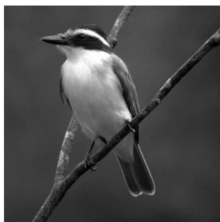
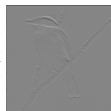


Imagen original



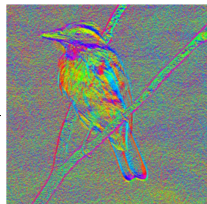
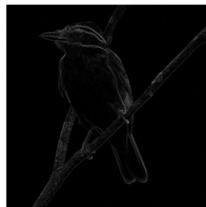
Filtro gaussiano



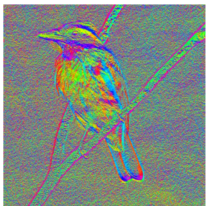
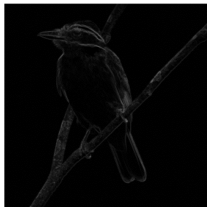
Derivadas



Magnitud



Dirección



Supresión no máxima

Umbralización por histéresis



Cierre de contornos

Algoritmo de Canny

- Obtención del gradiente: obtener magnitud y orientación del vector gradiente en cada pixel.
- Supresión de no máximos: Conseguir el adelgazamiento del ancho de los bordes, obtenidos con el gradiente, hasta lograr bordes de un pixel de ancho.
- Histéresis de umbral: Aplicar esta función de histéresis que está basada en dos umbrales; se quiere reducir la posibilidad de bordes falsos.

Obtención del gradiente

Aplicar un filtro Gaussiano para eliminar ruido y suavizar la imagen gradiente en cada pixel.

(a)

$$\frac{1}{273}$$

1	4	7	4	1
4	16	26	16	4
7	26	41	26	7
4	16	26	16	4
1	4	7	4	1

(b)

$$\frac{1}{115}$$

2	4	5	4	2
4	9	12	9	4
5	12	15	12	5
4	9	12	9	4
2	4	5	4	2

Máscaras Gaussianas sugeridas para suavizar que se obtiene promediando los valores de intensidad de los pixels en el entorno de vecindad con una máscara de convolución con un cierto σ .

Algoritmo para obtener el gradiente

Entrada: I imagen, máscara de convolución H .

Salida: I_m magnitud del gradiente, I_o orientación del gradiente

1. Suavizar la imagen I mediante el filtro Gaussiano H y obtener la imagen de salida J .
2. Para cada pixel (i, j) en J , obtener la magnitud y orientación del gradiente.
3. Obtener I_m e I_o

Supresión no máximos del gradiente

- Entradas I_m e I_o .
- Las orientaciones son: 0° , 45° , 90° , y 135° con respecto al eje horizontal.
- Para cada pixel se encuentra la dirección que mejor aproxime a la dirección del ángulo de gradiente.
- Si el valor de la magnitud de gradiente es menor que al menos uno de sus dos vecinos en la dirección del ángulo obtenida en el paso anterior, entonces se asigna el valor 0 a dicho pixel, sino se asigna el valor que tenga la magnitud del gradiente.
- La salida de este segundo paso es la imagen I_n de bordes, es decir, $I_n(i, j)$, después de la supresión no máxima de puntos de borde.

Algoritmo: Supresión no máximos

Entrada: I_m e I_o

Salida: I_n

Dadas cuatro direcciones d_1 , d_2 , d_3 y d_4 que representan a 0° , 45° , 90° , y 135° con respecto al eje horizontal.

- 1. Para cada (i, j) :
 - 1.1. Encontrar la dirección d_k que mejor se aproxima a la dirección $I_o(i, j)$, o sea la perpendicular al borde.
 - 1.2. Si $I_m(i, j)$ es menor a al menos uno de sus dos vecinos en la dirección d_k , al pixel (i, j) de I_n se le asigna el valor 0 o sea $I_n(i, j) = 0$ (supresión), sino $I_n(i, j) = I_m(i, j)$.
- 2. Devolver I_n

3. Umbral por histéresis

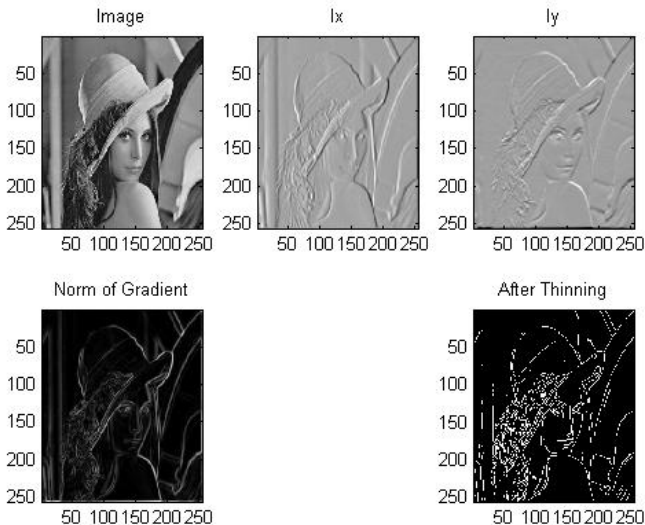
I_n suele contener máximos locales creados por el ruido. Para dar solución y eliminar dicho ruido se calcula el umbral por histéresis:

- considerar la imagen obtenida del paso anterior.
- considerar la matriz de orientaciones de los puntos de borde de la imagen.
- tomar dos umbrales U_{min} y U_{max} .
- para cada pixel de la imagen, si el valor de la magnitud del pixel considerado es mayor a U_{max} , entonces ese pixel es marcado como borde.
- se considera el siguiente pixel vecino no explorado que está en la dirección perpendicular al gradiente y que sea mayor a U_{min} y se lo marca como borde.
- a partir de dicho pixel seguir las cadenas de máximos locales conectados en ambas direcciones perpendiculares a la normal del borde, siempre que sean mayores a U_{min} .
- se marcan todos los pixels explorados y se almacena la lista de todos los pixels en el contorno conectado. Es así como en este paso se logra eliminar las uniones en forma de Y de los segmentos que confluyan en un pixel.

Cerrar los contornos

- consiste en cerrar los contornos que pudiesen haber quedado abiertos por problemas de ruido.
- método muy utilizado es el algoritmo de Deriche y Cocquerez. Este algoritmo utiliza como entrada una imagen binarizada de contornos de un pixel de ancho.
- el algoritmo busca los extremos de los contornos abiertos y sigue la dirección del máximo gradiente hasta cerrarlos con otro extremo abierto.

Ejemplos de Canny



Links

- <http://links.uwaterloo.ca/Repository.html>
-